

QK2 Component Map

A **provisional** component map and implementation hypotheses — a UI-free crypto-core boundary, the physical / software-privilege / logical distinctions, and the modules' *reference* fixtures. This is not yet a buildable realization, and no module has passed production conformance.

PROJECT STATUS — REFERENCE VECTORS, NOT PRODUCTION CONFORMANCE

Every "PASS" below means a **reference vector was generated or cross-checked by embit** (evidence levels 1–2) — *not* that production Rust/JavaCard code exists, runs on target, or survives a failure corpus. Module interfaces are hypotheses; the constants for several are blocked on open decisions. No target validation has occurred.

BOUNDARY KINDS	CORE MODULES	CORE LANGUAGE	FIXTURES	APPLET	DATE
physical · sw-priv · logical	UI-free selected set	Rust core + pinned libsecp FFI · C fallback	7 · ref / x-chk only	JavaCard	2026-08-17

1. [The organizing principle](#)

2. [The module map](#)

3. [The core/shell contract](#)

4. [Data flow: restore & sign](#)

5. [Conformance wiring](#)

6. [Repo layout & build](#)

One boundary the whole design rests on

All three reference codebases converge on the same architecture: a **UI-free, headless-testable crypto core** that takes bytes in and produces bytes and decisions out, wrapped by a shell that owns pixels, GPIO, and files and holds no key material by API discipline — *not* by isolation: same-process code is not isolated from the core (see §03). This is not a style preference — it is what makes the security-critical code testable (the QK2 reference vectors exercise a mock-driven reference model, not a production core, which does not yet exist) and what keeps secrets off the surfaces an attacker reaches.

THE CORE/SHELL BOUNDARY IS A SOFTWARE-DESIGN BOUNDARY, NOT A TRUST BOUNDARY

The core never does I/O (no display, GPIO, filesystem, clock, network) — it reaches the card and entropy sources only through injected interfaces, which is why it is headlessly testable. But **opaque handles and C APIs do not make the shell untrusted**: if the core and shell run in one process (or the same privileged Linux environment), a compromised shell can inspect process memory, forge an approval call, or render a report different from the one signed. The phrase "the shell never holds key material" describes an *API discipline*, not an enforced isolation.

Three *kinds* of boundary — kept distinct

- **Physical trust domains**: the terminal SoC, the smart card, and any truly independent recovery medium. These are the real trust boundaries.
- **Software privilege domains**: the kernel, the trusted qk-core wallet process, the unprivileged qk-io transport broker, the qk-decoy calculator, and boot/update components. Separation here requires actual OS mechanisms — separate processes, minimal IPC, privilege dropping, filesystem/device permissions, seccomp — and even then the kernel and booted image are trusted.
- **Logical module boundaries**: parser, descriptor engine, renderer, card client. These are design

seams, not security guarantees.

v1 **specifies** the process split rather than leaving it optional: `qk-core` owns the keypad, framebuffer, card sessions, and all secret memory during wallet mode and reads the physical approval directly; `qk-io` is an unprivileged transport broker (a selected SD file or camera-frame stream, a first outer size/time/frame cap, minimal IPC to `qk-core`) with no card access, no keypad-during-approval, and no secret API; `qk-decoy` is the calculator, which has no wallet/card APIs and is terminated before `qk-core` takes exclusive control. Process isolation limits accidental and low-privilege compromise; it does not turn a compromised kernel into a safe platform. The card applet is the one component on genuinely separate silicon; the bootloader is a root of trust *only as far as the CMO allows* — which, per the Blueprint, is not an immutable hardware root.

02 THE MAP

Modules by decision maturity

The settled product is a native **P2WSH** `wsh(sortedmulti(2, A, B, C))` 2-of-3: **A** = a Recovery Document (an ordinary printed webpage whose footer resembles the source URL and carries authenticated ciphertext of 256-bit Seed-A), **B** = a bus-pass Key Card, **C** = an Emergency/Inheritance Card, plus an **interchangeable** retro-calculator terminal that holds no persistent wallet secret. **Stealth, minimalism and hiding-in-plain-sight are a SYSTEM requirement** (artifact, document format, UI, labels, packaging) — a first layer of defence, *never* a cryptographic factor and never a substitute for the 2-of-3 threshold. An earlier revision described a *different* product (a 2-of-2 vault-key model); the modules below are grouped by **decision maturity**, and every entry is **selected product direction** carrying reference-computation / cross-check evidence only (levels 1–2) — none is a validated implementation.

update-verify

Signature check on signed offline update packages (detects accidental/unauthorized packages in the supported flow); indexed/revocable keys. **Not an immutable CM0 root of trust**; any anti-rollback floor needs a separately-protected monotonic store, unproven; physical reflash remains a residual risk.

Blueprint L1/\$1.3

(a) Approved v1 components

approved decisions · reference vectors only · Rust core + libsecp FFI / Linux

◆ **APPROVED DECISIONS – REFERENCE COMPUTATION / CROSS-CHECK ONLY (LEVELS 1–2); NO CODE RUNS ON TARGET · "NO DYNAMIC ALLOC" IS A TARGET NEEDING A MEMORY PLAN, NOT AN ASSUMPTION**

crypto-prim

Pinned libsecp256k1 (narrow FFI), SHA-256/512, HMAC, HKDF-SHA256, ChaCha20-Poly1305 (RFC 8439) AEAD. Terminal ECDSA is exact deterministic RFC6979 with low-S, verify-before-release (low-R optional, size-only; card nonce behavior is a Gate-B capability — NOT "hedged"). Constant-time; context re-randomised per session.

Blueprint L2

bip32-bip39

HD derivation (index bounds), mnemonic ± checksum, PBKDF2-2048 with NFKD. **Default 24 words / 256-bit for A/B/C** (not 12, not "advanced"). A2 is a separate 256-bit random key, NOT a BIP-39 signing key.

Standards Register

entropy-engine

Provisioning-only, 256-bit. **Domain-separate the conditioner (QKEC-1, HKDF-SHA256)** for Seed-A / Signer-B / Signer-C / A2 with fresh acquisition each — do NOT derive B/C or A2 from A or one master. TWO production modes ONLY: (1) **Recommended: Multi-source** = terminal HW source + card-B RNG + card-C RNG (+ optional user physical), mixed via HKDF; (2) **Advanced: pure physical** = exactly 100 private d6 rolls per secret (400 total), SHA-256 of the literal 100 ASCII digits, no HWRNG reliance — 99 rolls is NOT the threshold. CloakGrid stays research-only

psbt-engine

Bounded PSBT v0 parser: streaming parse, validate (fatal/warn), fee review (absolute fee AND fee rate), approval binding (an immutable canonical review-object identity, re-checked immediately before signing), sign. Unknown fields preserved unless an explicit documented rejection policy applies.

PSBT Checklist

(experiments/), NOT a production entropy mode. ColdCard-2026 lesson: PROVE the intended source reaches the conditioner in the release binary (reachability + fault injection + reproducible pure-physical mode) — health tests alone can't.

QKEC-1

renderer

ST7789 SPI framebuffer; pure screen functions; full-address chunked layout; framebuffer wipe. Security-critical — the report it shows can mislead the user, so a compromised renderer can lie.

Blueprint L8

microsd-io

microSD is the **lower-risk FIRST transport** — QR is ALSO required (see qr-transport). **Decide the FS stack first:** Linux kernel FAT mount, or a userspace FAT lib over a raw device? "Hardened FatFs" applies only to the latter. Read-only intake, copy-in, never-overwrite; the signed result is a new object (temp → sync → validate → rename → cleanup).

PSBT Checklist S0/S9

ui-fsm

Destination/back-stack state machine, security flush points, dead-end fraud screens, scroll-gate.

PSBT Checklist S8

input

MCP23017 keypad matrix over I2C; reject-invalid-at-entry for dice.

Blueprint L0

secret-lifecycle

locked, non-swappable static buffers; memset-on-drop with compiler barrier; wipe triggers.

Blueprint L9

card-transport

SEC1210 over UART, ISO-7816 T=1 APDU framing. A dumb pipe needing integrity/error handling, NOT a secret channel — in v1 the card bus is inside the trusted terminal boundary; no custom secure channel, no fleet key, no per-terminal pairing.

Card Protocol

power-mgmt

ADS1015 battery ADC, GPIO power sequencing, low-battery signing gate.

Blueprint L0

calculator-decoy

the disguise app; the only thing reachable card-absent; leaks no wallet state. Camouflage is a system requirement, never a crypto factor.

Blueprint L8

a2-lifecycle

A2 = a FRESH 256-bit random key generated at provisioning, DUPLICATED onto BOTH cards B and C, verified, then WIPED from the

a1-aead-decrypt

TERMINAL-SIDE AEAD decrypt of the A1 recovery-document ciphertext using A2 released from a bearer card. **ChaCha20-Poly1305**

card-client

Role-aware minimal card session, SAME applet for roles B and C: GET_INFO / GET_POLICY / GET_A2 / SIGN /

pair-session

Factor-PAIR session controller: loads exactly ONE pair — A1+B / A1+C / B+C — reconstructs+verifies D, validates vs the 2-of-3 policy, and

terminal. A2 decrypts the A1 document; it is NOT a Bitcoin key and is EXPORTABLE from a bearer card by design.

[Provisioning \[SELECTED\]](#)

(RFC 8439); document key = HKDF-SHA256(IKM=A2, salt=wallet_id, info="QuietKey/A1/v1"); fresh stored 96-bit nonce; 128-bit tag; NO plaintext before tag verification. The permanent document format is NOT coupled to whatever AEAD one JavaCard exposes.

[Recovery Document \[SELECTED\]](#)

PROVISION_BEGIN·DATA·COMMIT·ABOR / ATTEST (optional, Gate-B). NO custom secure channel in v1 — the bus is inside the trusted terminal boundary; NO fleet key, NO single-terminal pairing; a standard card-authenticated profile (e.g. SCP11) is only optional LATER hardening. Retirement is on-chain sweep + physical destruction, NOT an applet wipe command. Terminal interchangeable.

[Card Protocol \[SELECTED\]](#)

produces the TWO signatures of the loaded pair. Physical custody of a pair = authorization.

[Wallet Flows \[SELECTED\]](#)

address-pack

Exports individual P2WSH RECEIVE addresses / packs for online monitoring — the **Quantum Shelter** (address-only) path. Simple Recovery & Inheritance tiers MAY instead export the full watch-only descriptor to a supported coordinator (with the privacy / future-pubkey-exposure tradeoff); address-only is NOT a universal rule. No address / child-key reuse. D stays offline + encrypted at rest.

[Descriptor D \[SELECTED\]](#)

quantum-shelter

Descriptor-confidentiality / Deep-Cold mode: D kept as a separate cloaked/encrypted "wallet map"; online sees addresses only. The signed output is a FINALIZED RAW TRANSACTION — no PSBT xpub/origin metadata goes back online.

[Kit Tiers \[SELECTED\]](#)

pq-migration

Quantum-PREPARED not quantum-proof: 256-bit seeds, dormant P2WSH keys/descriptors offline, no address reuse. Reserve versioning + benchmark larger PQ payloads; sweep from hidden-key P2WSH when a reviewed PQ path deploys. Do NOT freeze a proprietary seed-to-PQ derivation or ship an off-chain PQ-signed PSBT.

[Quantum \[SELECTED\]](#)

(b) Selected wallet-model components

native P2WSH 2-of-3 selected · format constants not yet frozen

descriptor-engine

Native P2WSH `wsh(sortedmulti(2, A, B, C))` — **SELECTED**. `sortedmulti` (BIP-67), `owns()` change proof, address derive, BIP-380 checksum. 2-of-3 with independent A/B/C is the PRIMARY

recdoc-codec

Recovery-document print codec. **Crypto capsule is FROZEN**: ChaCha20-Poly1305 (RFC 8439), fresh stored 96-bit nonce, 128-bit tag, payload = version | nonce | ciphertext(32) | tag(16), wallet_id

camera-ocr

CSI capture with TWO mode-locked uses ONLY — A1 footer capture and PSBT QR capture; NEVER auto-detect a seed QR or accept arbitrary QR classes. Feeds recdoc-codec as erasures. Confidence thresholds

keyC-ceremony

Emergency Card C provisioning — the terminal derives an independent **account xprv at m/48'/0'/0'/2'** (+ chain code + origin fingerprint) from an independent Seed C, loads THAT (NOT a root mnemonic) plus the

policy; 2-of-2 is a historical/regression fixture only. D is recovery-critical AND quantum-sensitive AND privacy-sensitive (xpubs expose EC pubkeys). A checksum is error-DETECTION not authentication — at provisioning derive+compare every A/B/C origin/xpub from the ACTUAL factors and rehearse scan-back.

[Descriptor D](#) [SELECTED]

bound as AAD. Only the **physical layer stays Gate-A** — RS over GF(256), OCR-safe base32, footer length/font/grouping/OCR thresholds. The old RS(83,49) / 49-byte / 133-sym / 142-char layout is NOT normative — recompute after realistic print/capture trials. Footer resembles the page's source URL; NO naked BIP-39, no crypto label.

[Recovery Document](#) [Gate-A physical layer]

await the Gate-A capture study. Wipes DMA buffers.

[Recovery Document](#)

SAME A2 + recovery metadata per kit tier, then wipes the root. 2-of-3 is SELECTED (no longer contingent). C is a BEARER card, NO PIN, must work after years. Kit tiers customize packaging/instructions/D-placement, not the crypto.

[Wallet Policy](#) [SELECTED]

(c) Selected but validation-blocked

on-card primitives bench-required (Gate-B) · QR optics blocked (Gate-A)

card-signer (B/C)

SAME minimal applet in two roles — Key Card B and Emergency Card C — each holding an independent **account xprv (m/48'/0'/0'/2' + chain code + origin fingerprint)**, **NOT a root seed/mnemonic**; the applet does NOT implement BIP39/PBKDF2 and derives only non-hardened receive/change children. Signer key NEVER exportable; A2 exportable by design. Bearer objects: physical custody = authorization, NO PIN/OAC/lockout. On-card ECDSA with low-S, verify-before-release; card nonce behavior (NOT "hedged") is a Gate-B capability to specify/test — bench-required on the target card.

[Card Protocol P4](#)

provision-fsm

Provisioning ceremony: generate A2 with Seed-A, write the SAME A2 to B and C, verify both, then WIPE A2 + all seeds from the terminal. No preinstalled secret; terminal interchangeable. Card primitive — bench-required.

[Card Protocol P6](#)

attestation

Manufacturer-CA identity; challenge-response. A MANUFACTURING-security question, NOT user authentication — only with a real trust chain + rotation/revocation. Monotonic counters are not clone-detection without an external anchor + user action. Bench-required.

[Card Protocol P1](#)

qr-transport

QR import/export — REQUIRED alongside microSD (microSD is the lower-risk first path). **BBQr (file type P, uncompressed base32) is the ONE selected v1 PSBT framing** — no BC-UR/UR2/Specter/SeedQR, no generic autodetect. Capture/animated-reassembly/screen-export/RAM/performance on the existing enclosure/optics/320×240 display remain a Gate-A HW/optics BLOCKER — NO mechanical redesign authorized — but the FORMAT is settled, not pending a bake-off.

[Transport](#) [REQUIRED · BLOCKED]

(d) Rejected v1 alternatives

experiments/ only · short appendix — NOT mainline

oac-gate + card-PIN

REJECTED. HMAC authorization, escalating lockout, retry-counter, card PIN, duress, computational-delay, bricking. B and C are bearer objects — the 2-of-3 policy is the spending control. Matters most for the inheritance kit (must work after years with no forgotten PIN/vendor/account/clock).

rejected

custom-card-channel

REJECTED for v1 (sc-server + the custom ECDH → AES-GCM / HMAC channel). In v1 the card bus is inside the trusted terminal boundary — no channel at all. A standard card-authenticated profile (e.g. SCP11, no per-terminal pairing / shared host secret) is only optional LATER hardening, never a recovery dependency.

rejected

vault-key RELEASE-SEED

REJECTED. Primary-card-only Vault-Key that releases Seed-A, and on-card capsule AEAD (qkrd-capsule). A2 is duplicated to BOTH bearer cards and the AEAD decrypt runs on the replaceable terminal — no card-side release-seed / vault-key dependency.

rejected

miniscript-timelock

REJECTED for the vault (BIP-379 Draft). Timelock, cloned-B, encrypted-B-export and single-sig are all rejected wallet alternatives — native P2WSH 2-of-3 is the selected model.

rejected

anti-exfil

DEFERRED (not rejected, not a v1 blocker). Caller-controlled-nonce sign-to-contract; a one-shot JavaCard ECDSA likely cannot do it. Optional future hardening.

deferred

03 THE CONTRACT

The seams — what crosses the boundary

The core exposes pure entry points and depends on the shell only through two injected interfaces (a card operations vtable and an entropy-source vtable). Those two injection points are exactly what the fixtures replace with mocks — the reason the reference model is headlessly testable. The vectors exercise a reference model only; no production core exists.


```
// ---- core entry points (shell calls down; pure, deterministic) ----
Ent      entropy_encode(Mode mode, const Source* srcs, size_t n);      // QKEC-1
Master   hd_master(const Seed*);   Addr descriptor_address(const Desc*, u8 branch, u32 i);
bool      descriptor_owns(const Desc*, const Output*);                  // change proof
RecoveryPayload recdoc_decode(const u8* syms, size_t n);                // -> RecoveryPayload | REFUSED; capsule crypto FROZEN (ChaCha20)
Result    psbt_parse(const u8* buf, size_t len, PsbtHandle* out);        // bounded; explicit error enum, owned handle
Result    psbt_validate(PsbtHandle, const Desc*, Report* out);          // no naked Psbt* - lifetime/ownership explicit
Result    psbt_sign(PsbtHandle, FactorPair pair, const Desc*, const CardOps*, Signed* out); // load pair {A1+B|A1+C|B+C}, reconstruct+
// pointer-return APIs (Psbt*) are replaced with Result enums + bounded owned handles

// ---- injected interfaces (shell implements; core depends only on these) ----
struct CardOps { // real = SEC1210 transport; test = mock/embit; SAME minimal applet for card roles B and C
    Info      (*get_info)(Session);                // role (B/C) + firmware; no secret
    Key        (*get_a2)(Session);                  // A2 EXPORTABLE by design (a bearer card releases the A1 decryption key)
    Sig        (*sign)(Session, const Sighash*);     // signer key NON-exportable; on-card ECDSA (low-S, verify-before-release)
    Policy     (*get_policy)(Session);               // minimal wallet-policy metadata
    Status     (*provision)(Session, const Provision*); // write account xprv (m/48'/0'/0'/2') + shared A2 + policy; NO BIP39/PBKDF2 on-
    Sig        (*attest)(Session, const Challenge*); // optional (Gate-B) manufacturer-CA attest; NO retire-wipe primitive - retireme
    // no release_seed(Capsule) primitive; no custom secure channel / fleet key / pairing; no anti-exfil / Rho parameter; no PIN/OAC/lock
};
struct EntropySource { size_t (*draw)(u8* out, size_t n); Health (*health)(void); };

// ---- secret handling: opaque to the shell ----
SecretRef secret_alloc(void); void secret_wipe(SecretRef); // locked static buffers; no heap copies
```

LANGUAGE — A DECISION RECORD, RUST-FIRST

Rust is the main implementation hypothesis for the new core, parsers, state machines, recovery codec, and transport decoding (PSBT, descriptor, OCR, SM are hostile-input parsers where memory safety is a direct security property), **gated on the exact ARMv6/Linux target toolchain passing Gate C**. Use a **narrow, pinned C FFI ONLY for Bitcoin Core's libsecp256k1**. Rust does not magically zeroise — secret zeroization still needs deliberate engineering (fixed static buffers, `memset -on-drop` behind a compiler barrier, `mlock`). If target measurements rule Rust out, the fallback is a written decision record for an all-C core with checked arithmetic, fixed arenas, whole-program fuzzing, ASan/UBSan host builds, static analysis, compiler hardening, and a

formal memory budget. **Python and JavaScript are NOT acceptable at the production secret boundary.** "No dynamic allocation" is a target to demonstrate with a memory plan, not an assumption.

04 DATA FLOW

Two operations across the modules

Decode the A1 recovery document

```
// A bearer card releases A2; the REPLACEABLE TERMINAL performs the AEAD decrypt – no card-side release-seed / vault-key
camera-ocr -(A1 footer syms+conf)→ recdoc-codec -(payload)→ a1-aead-decrypt ←(A2)– card-client → card-signer (GET_A2)
  ▲ shell                                ♦ core                                ♦ core (stored random nonce)    ♦ core      applet · A2 exportable
    └ erasures, never guesses              └ fail-closed                      └ recovers 256-bit Seed-A entropy
[Seed A recovered] → bip32-bip39 → descriptor-engine -(addr)→ renderer
                        ♦ core              ♦ core                                shell
secret-lifecycle holds Seed A / A2 in locked buffers; wiped on card removal/timeout.
```

Sign a PSBT — load a factor PAIR

```
microsd-io / qr-transport -(bytes)→ psbt-engine S0-S7 -(Report)→ ui-fsm -(scroll-gated approve)→ pair-session
  shell                                ♦ core                                shell                                ♦ core
    | reconstruct + verify D, validate vs 2-of-3 (checksum ≠ auth: re-derive origins from the loaded factors)
    | change proof via descriptor-engine.owns(); bind_hash (anti-TOCTOU)
    └ fatal ⇒ dead-end screen, discard
load ONE pair {A1+B | A1+C | B+C} → produce the TWO signatures of the loaded pair:
  · A1 leg: a1-aead-decrypt (A2 from a bearer card) → crypto-prim (deterministic RFC6979 + low-S)
  · B/C leg: card-client → card-signer (on-card ECDSA; bench-required; anti-exfil deferred)
```

CORRECTION — THE SHELL CAN CAUSE A SIGNATURE

An earlier version claimed "the shell can never cause a signature." That is false on the current single-CPU design: a shell that can call `psbt_sign()` can cause a signature. What the core actually guarantees is that **the signed transaction is revalidated in the core after the UI returns approval** (the S9 re-check with no interactive yield). It cannot, on its own, guarantee the user *saw* what was signed — **the report the core hands the renderer is security-critical, and a compromised renderer can lie** even when the core validates correctly. Binding a trusted approval to what the user saw needs an independently authenticated physical approval signal and core control of the display path — which v1 gives `qk-core` (it owns the framebuffer/keypad in wallet mode). Even with the three-process split, the kernel and booted image remain trusted: process isolation limits accidental/low-privilege compromise, it does not turn a compromised kernel into a safe platform, and a single malicious terminal in an A1+B session can still see Seed A + A2 and obtain the normal threshold.

05 CONFORMANCE

Every core module has a reference fixture

Each fixture was run to produce this document — but that establishes only **reference-vector generation and embit cross-check (levels 1–2)**, shown in the one filled column below. The columns that decide whether a *module* passes are empty: **Rust core implementation (C fallback) · JavaCard implementation · target integration · independent implementation · negative/fault tests · recovery rehearsal**. No production module passes until its actual code reproduces the vector *and* survives the failure corpus. Wire the reference fixtures into CI as a floor, not a finish line.

			EVIDENCE
crypto-prim · bip32-bip39	qkec1.py	BIP-39 Trezor vector · RFC 5869 HKDF · secp256k1 73c5da0a	REF / X-CHK
entropy-engine	qkec1_reference.py	dice TV-A1/A2 · hybrid TV-B1/B2 · seed 5eb00bbd	REF / X-CHK
descriptor-engine	qk2_descriptor.py	BIP-32 TV1 · bech32 BIP-173 addr · 2-of-3 (independent A/B/C) is the PRIMARY policy vector — fingerprints A=2fae9711 B=72a14ab8 C=badaa8ff, checksum #kuyqfyj3, recv/0/0 bc1qecahkw0srs8xpmgwqya49qtknxwvkwrt0ky4jds52scsw4etmd6s5s93pr (reference computation, not canonical); 2-of-2 #w9nvs8vx is a historical/regression fixture only	REF / X-CHK
psbt-engine	qk2_psbt_corpus.json	1 baseline (must-sign) + 6 attacks (must-reject), embit oracle	REF / X-CHK
recovery descriptor (2-of-3)	qk2_recovery_corpus.json	selected 2-of-3 recovery metadata (all A/B/C xpubs + origins, two checksummed receive/change descriptors as the conservative authority); the timelock-path fixture is REJECTED (miniscript BIP-379 Draft)	REF / X-CHK
card-client	qk2_card_protocol.py	role-aware minimal protocol (GET_INFO/GET_POLICY/GET_A2/SIGN/PROVISION/ATTEST); NO custom secure channel in v1 (card bus inside the trusted terminal boundary); custom ECDH channel REJECTED; anti-exfil deferred · bench-required	REF / X-CHK
recdoc-codec	qk2_recdoc.py	RS(83,49) round-trip · erasure recovery · fail-closed — a HISTORICAL/regression codec fixture; the settled capsule crypto is FROZEN (terminal-side ChaCha20-Poly1305, 96-bit nonce, 128-bit tag) and only the RS/base32/OCR physical layer is Gate-A, to be recomputed after realistic print/capture trials	REF / X-CHK

Shipped as `qk2_conformance_manifest.json` . Beyond these, the intake boundaries (`psbt_parse` , `recdoc_decode` , the SM parser) need **fuzz targets** off-device, and the shell needs SeedSigner-style

headless flow tests driving the ui-fsm against the real core.

06 BUILD

Repo layout & posture

```
qk2/
├─ core/          qk-core (trusted, Rust main hypothesis): entropy BIP39/32 descriptors A1 PSBT policy review – crypto-prim bip32-bip39
├─ include/       the entry points + CardOps/EntropySource vtables (§03) – the ONLY public v1 surface; rejected card-channel constructs
├─ secp/          pinned libsecp256k1 + minimal reviewed FFI – the ONE narrow C boundary
├─ applet/        JavaCard, one minimal applet in roles B and C, bench-required: card applet + APDU spec (get_info get_policy get_a2 s:
├─ platform/      trusted framebuffer / keypad / card adapters (renderer input card-transport power)
├─ io-broker/     qk-io (unprivileged transport broker, NO secrets / NO card access): camera-ocr microsd-io qr-transport (BBQr); first
├─ decoy/         qk-decoy calculator app; no wallet/card APIs; terminated before qk-core takes the display/keypad/card slot
├─ rescue/        open CCID/ISO-7816 recovery CLI + reproducible rescue image (commodity reader; NOT QuietKey-hardware-only)
├─ simulator/     host simulator: reference model + mock cards/keypad/display/camera/SD – runs the fixtures + headless ui-fsm flow tes
├─ fixtures/      normative vectors + hostile corpora: qkec1 · descriptor · psbt corpus · recovery · card · recdoc · manifest
├─ fuzz/          PSBT · descriptor · BBQr · A1 · APDU · state-machine targets
├─ docs/          threat model · protocol specs · recovery runbooks · SBOM
├─ third_party/   pinned dependencies + licenses (clean-room; NO Commons-Clause COLDCARD code copied in)
├─ bootloader/    signed-image verify (anti-rollback = candidate – needs a separately-protected monotonic store)
├─ experiments/   OUTSIDE core/, NOT mainline v1: CloakGrid · custom ECDH card channel · sc-server · oac-gate + card-PIN · vault-key RI
```

BUILD POSTURE (FROM THE SPECS)

Reproducible builds with published digests (the iancoleman single-hashable-artifact discipline). **Boot straight to the app**, no desktop. Gate-A reader constants and the candidate protocol constants are **build-controlled and CI-checked**. The core builds and tests on a host with zero hardware — that is the whole point of the boundary.

MODULES BLOCKED ON OPEN VALIDATION (MODEL IS SETTLED)

The product model is settled (native P2WSH 2-of-3, ChaCha20-Poly1305 A1 capsule with A2 duplicated to both bearer cards, BBQr + microSD, account-xprv cards, no PIN/OAC/pairing/secure-channel), but several modules cannot be finalised until validation items close: **recdoc-codec / a1-aead-decrypt** — the crypto capsule is FROZEN (ChaCha20-Poly1305, 96-bit nonce, 128-bit tag); only the RS/base32/font/OCR physical layer + the Gate-A capture study on the REAL disguised printed webpage remain open; **qr-transport** — BBQr is the selected framing, but capture/RAM/performance on the existing enclosure (no mechanical redesign authorized) is still a Gate-A HW/optics BLOCKER; **entropy-engine** must PROVE the intended source reaches the conditioner in the release binary (ColdCard-2026 lesson) and its cutoffs await the CM0 HWRNG SP 800-90B assessment; the **card protocol** is minimal (no PIN/OAC/pairing) but its on-card primitives are bench-required. Constants are not yet frozen.

QK2 Module Map · working draft v1 · 2026-08-17

A **provisional** component breakdown of the QuietKey QK2 working drafts around the UI-free crypto-core boundary. The fixtures establish reference-vector and embed cross-check evidence only (`qk2_conformance_manifest.json`); **no module has passed production or target conformance**. Companion to — not a buildable realisation of — the QK2 Blueprint, QKEC-1, Standards Register, PSBT Checklist, Recovery Decision, Card Protocol, and Recovery Document.